MindCare Chat — Technical Documentation

Oct 11, 2026 · @Dhanushka Gunasinghe

1. Executive summary

MindCare Chat is a research prototype: a supportive mental-health chatbot built by fine-tuning **Qwen2.5-3B-Instruct** with **QLoRA** on 2,205 therapist answers from the public CounselChat dataset, served through a **Streamlit** web app on Google Colab.

Item	Detail
Purpose	Give empathetic, general coping information on everyday mental-health concerns (stress, anxiety, low mood, family conflict, self-esteem)
Users	People seeking supportive conversation; researchers evaluating small fine-tuned LLMs
Base model	Qwen/Qwen2.5-3B-Instruct (3.1 B parameters)
Fine-tuning	LoRA rank 16 on all attention and MLP projections; 29.9 M trainable parameters (0.96%)
Training data	CounselChat: 2,775 raw Q&A rows → 2,609 cleaned rows, split 80/10/10 by question
Interface	Streamlit chat UI, multi-turn memory of the last 10 messages
Safety	System prompt (no diagnosis, no prescribing) + keyword crisis filter that returns UK emergency guidance (999, A&E, Samaritans 116 123)
Hosting	Google Colab, Tesla T4 GPU, model in 4-bit; public URL via Cloudflare quick tunnel
Status	Working prototype, demonstrated end-to-end in an 11-minute screen recording

Scope boundary: MindCare Chat is not a therapist, diagnostic tool or emergency service. The app states this on screen, and the crisis path bypasses the model entirely.

2. Simplified technical overview

In plain terms: a general-purpose small language model was taught to answer like a counsellor by showing it about 2,200 real therapist replies, then wrapped in a chat web page with a safety net for crisis messages.

The project runs in four stages:

1. **Collect** — download the CounselChat dataset from Hugging Face into `data/raw` (`dataset.ipynb`).
2. **Prepare** — drop rows with missing question or answer text and split by question ID into train, validation and test CSVs, so no question appears in two splits.
3. **Train** — convert each row into a Qwen chat record (system prompt + user question → assistant answer) and fine-tune a 4-bit Qwen2.5-3B with LoRA adapters (`Untitled0.ipynb`). Only the small adapter (about 120 MB) is saved to Google Drive.
4. **Serve** — a Streamlit app loads the base model plus the adapter, screens each message for crisis terms, and generates a reply (`Untitled1.ipynb` → `app.py`). A Cloudflare tunnel exposes it on a public URL.

Technology stack

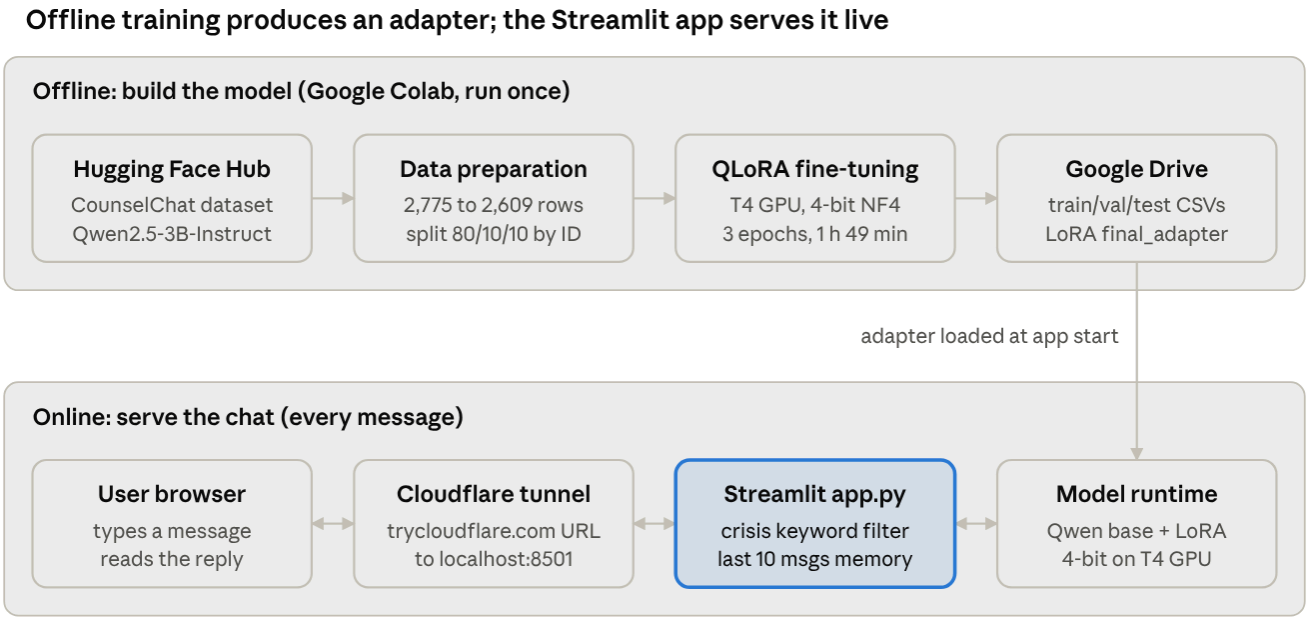
Layer	Technology	Version used
Language	Python	3.13 (Colab)
Deep learning	PyTorch + CUDA	2.11.0 + cu130
Model library	Hugging Face Transformers	5.19.0
Fine-tuning	PEFT (LoRA), TRL `SFTTrainer`	PEFT 0.21.2, TRL 1.15.0
Quantisation	bitsandbytes (NF4, double quant)	0.50.2
Multi-GPU / device map	Accelerate	1.15.0
Data	pandas, Hugging Face `datasets`	—
Web UI	Streamlit (`st.chat_input` , `st.chat_message` , `st.cache_resource`)	—
Tunnelling	Cloudflare `cloudflared` quick tunnel (LocalTunnel tried first)	2026.10.0
Compute	Google Colab, NVIDIA Tesla T4 (16 GB), Google Drive storage	—

Key parameters at a glance

Parameter	Training	Inference
Precision	4-bit NF4 weights, FP16 compute	4-bit NF4, BF16 if supported else FP16
Max length	1,024 tokens per example	220 new tokens per reply
Sampling	—	temperature 0.7, top-p 0.9, repetition penalty 1.1
Context	single Q&A pair	system prompt + last 10 messages

3. High-level architecture

The system has two halves: an offline pipeline that runs once to produce a LoRA adapter, and an online app that loads that adapter and answers every chat message.



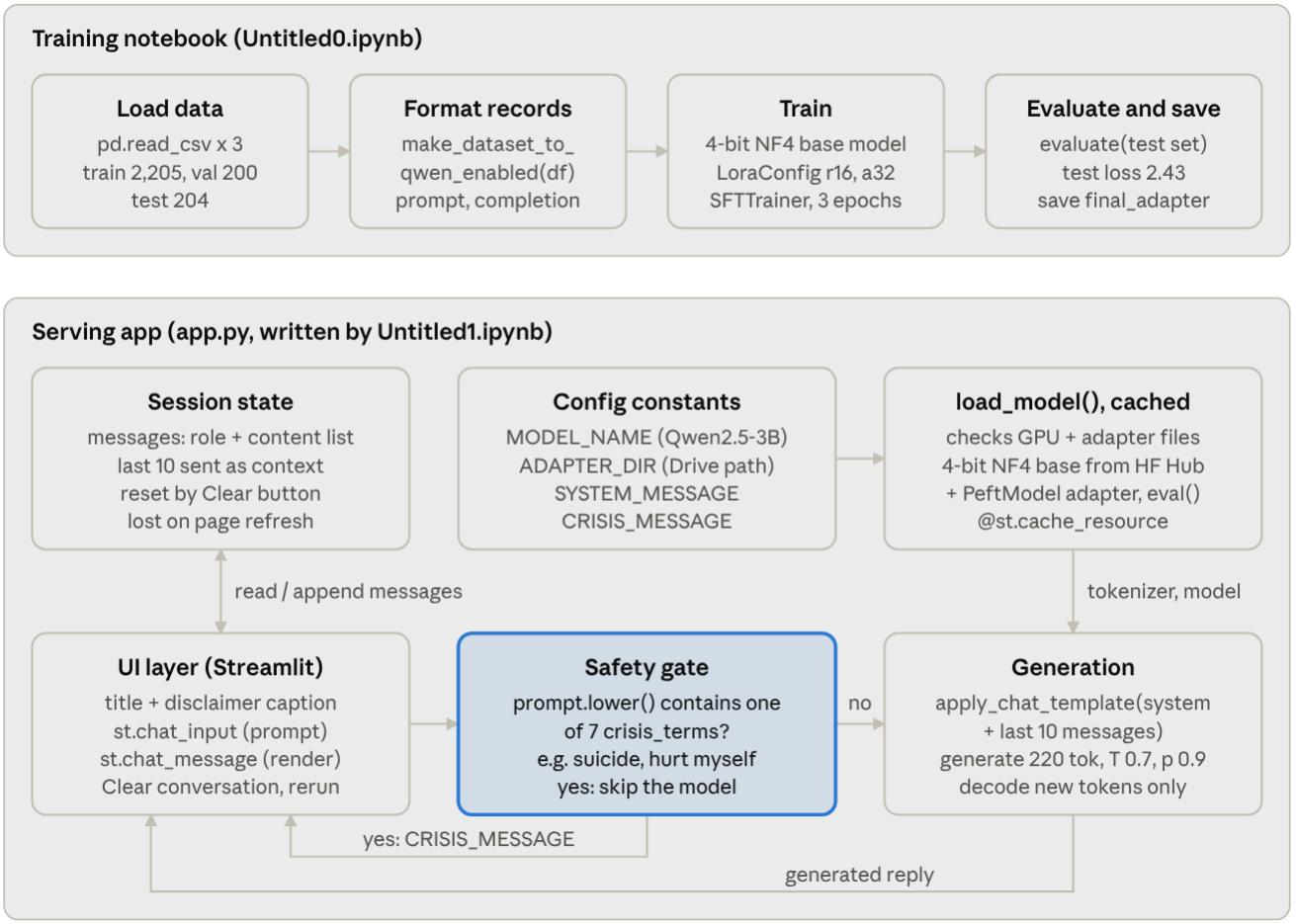
High-level architecture · offline build and online serving

The top row runs once and leaves the adapter on Google Drive; the bottom row runs on every message, with the Streamlit app deciding whether the model is called at all.

4. Low-level design

The training notebook is a straight four-step pipeline; `app.py` is six components, and the safety gate decides whether the model runs.

Every chat message passes the safety gate before it can reach the model



Low-level design · training notebook steps and app.py components

A crisis match returns the fixed `CRISIS_MESSAGE` straight to the UI; otherwise Generation builds the prompt from the system message and the last 10 turns and returns only the newly generated tokens.

Component reference (`app.py`)

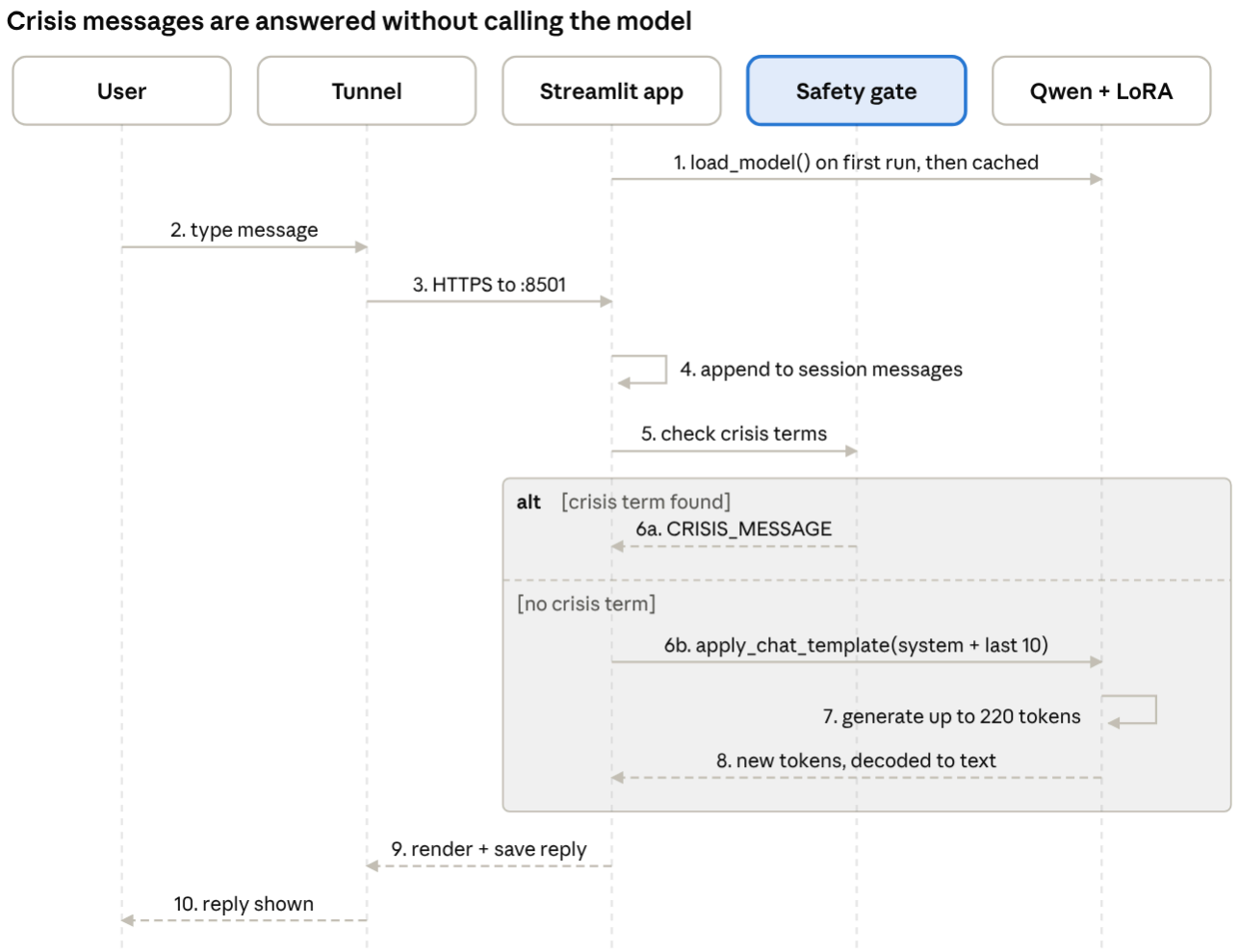
Component	Key code	Responsibility
Config constants	`MODEL_NAME` , `ADAPTER_DIR` , `SYSTEM_MESSAGE` , `CRISIS_MESSAGE`	Single place to change model, adapter path, behaviour rules and the crisis text
`load_model(adapter_path)`	`@st.cache_resource` , `BitsAndBytesConfig` , `AutoModelForCausalLM` , `PeftModel.from_pretrained`	Loads once per server process; fails fast with a clear error if there is no GPU or the adapter files are missing

Component	Key code	Responsibility
Session state	`st.session_state.messages`	Holds the conversation for the browser session; re-rendered on every Streamlit rerun
UI layer	`st.title`, `st.caption`, `st.chat_input`, `st.chat_message`, `st.button`	Shows the disclaimer, takes input, renders messages, clears history
Safety gate	`crisis_terms` list: kill myself, suicide, end my life, want to die, self harm, self-harm, hurt myself	Case-insensitive substring match; on a hit, skips the model and returns UK emergency guidance
Generation	`apply_chat_template`, `model.generate`, `tokenizer.decode`	Builds the prompt, samples up to 220 tokens under `torch.no_grad()`, falls back to a retry message if output is empty or an exception is raised

5. Sequence diagrams

5.1 Chat request

Each message takes one of two paths: a crisis match returns fixed UK safety guidance immediately, and any other message goes to the model with the last 10 turns as context.

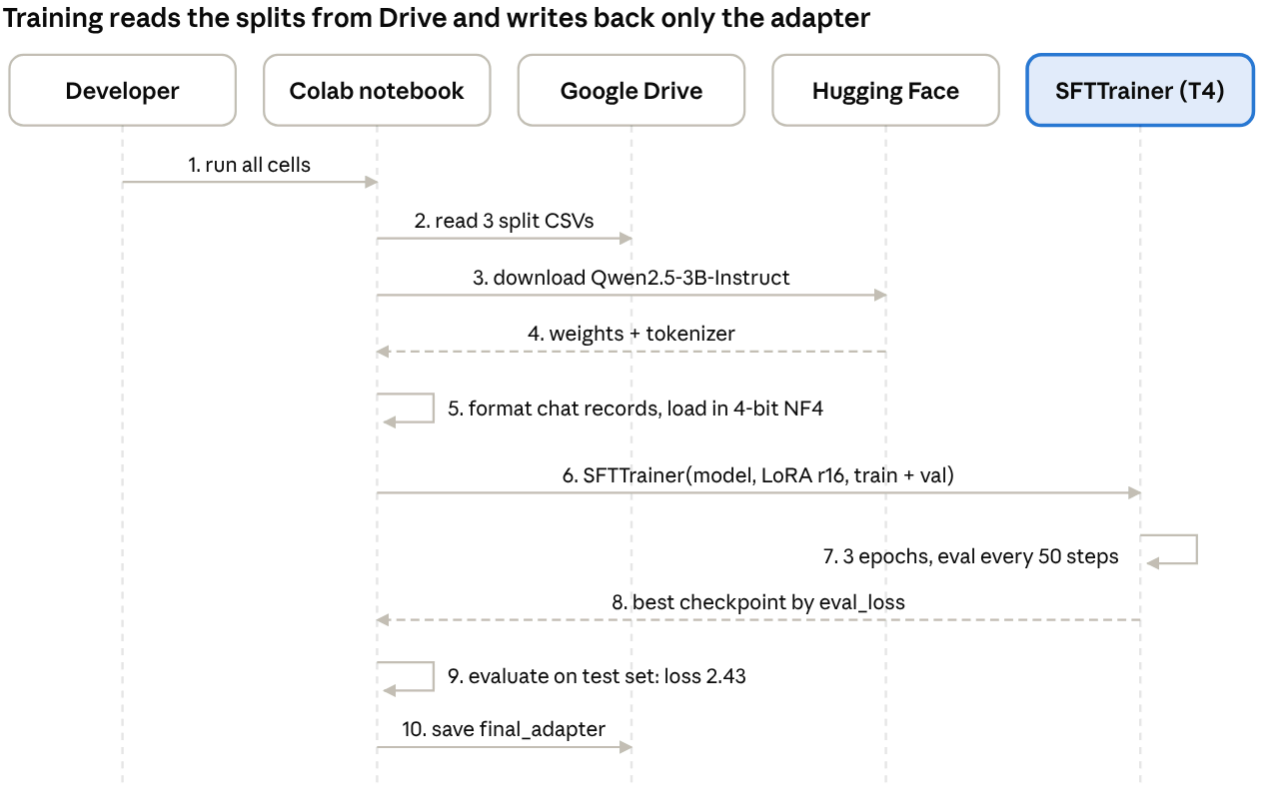


Sequence · one chat message, crisis and normal paths

Step 1 happens once per server start because `load_model()` is cached; every later message starts at step 2.

5.2 Training run

The training notebook pulls data from Drive and the base model from Hugging Face, trains on the T4, and saves only the small LoRA adapter back to Drive.



Sequence · training run in Colab

The best checkpoint (lowest validation loss) is reloaded before the test evaluation and the save.

6. Data: raw and processed

The raw CounselChat file holds 2,775 answers to 940 user questions; cleaning kept 2,609 answers to 862 questions, split 80/10/10 by question with zero overlap between splits.

Raw dataset — `counsel-chat.csv`

Each row is one licensed therapist's answer to a question posted on counselchat.com. One question usually has several answers (about 3 on average).

Column	Meaning	Used in training
`questionID`	Question identifier, 0–939	Split key
`questionTitle`	Short title of the question	No
`questionText`	Full question body (139 rows empty)	Yes — user message
`questionLink`	URL of the question	No

Column	Meaning	Used in training
`topic`	One of 31 topics	No (profiling only)
`therapistInfo` / `therapistURL`	Answering therapist	No
`answerText`	Therapist's answer (26 rows empty)	Yes — assistant message
`upvotes` / `views`	Engagement signals	No

Top topics by answer count: depression 465, anxiety 358, counseling-fundamentals 270, intimacy 248, relationships 202, parenting 191. Median length is 44 words per question and 136 words per answer.

Processed splits

File	Rows (answers)	Unique questions	Topics	Median answer words	Role
`train.csv`	2,205	689	29	138	Fine-tuning
`val.csv`	200	87	24	129	Eval every 50 steps, best-checkpoint selection
`test.csv`	204	86	19	150	Final held-out loss
Total	**2,609**	**862**	31	—	—

Processing rules (as observed in the outputs)

1. Remove rows with an empty `questionText` or `answerText` (2,775 → 2,612; 3 further rows dropped, likely blank strings).
2. Split by `questionID`, so all answers to one question land in the same split — this prevents the model being tested on a question it has already seen.
3. Keep all 10 original columns; training reads only `questionText` and `answerText`.

The split script itself was not among the uploaded files (`dataset.ipynb` only creates `data/raw`); the rules above are inferred from the CSVs.

Training record format

Each row becomes one prompt–completion pair in Qwen's chat format:

```
{
  "prompt": [
    {"role": "system", "content": "You are a supportive mental-health
information assistant. Respond with empathy and respect. Do not diagnose
people, prescribe treatment, or claim to replace a qualified professional.
Encourage appropriate professional support when needed."},
    {"role": "user", "content": "<questionText>"}
  ],
  "completion": [
    {"role": "assistant", "content": "<answerText>"}
  ]
}
```

7. Model training

Three epochs of QLoRA on a single T4 took 1 h 49 min and reached a held-out test loss of 2.43 (perplexity about 11.4) with 45.5% next-token accuracy.

Configuration

Group	Setting	Value
Quantisation	`BitsAndBytesConfig`	`load_in_4bit=True`, `nf4`, double quant, FP16 compute
k-bit prep	`prepare_model_for_kbit_training`	gradient checkpointing on
LoRA	rank / alpha / dropout	16 / 32 / 0.05
LoRA	target modules	`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`
LoRA	trainable parameters	29,933,568 of 3,115,872,256 (0.96%)
Trainer	`SFTTrainer` + `SFTConfig`	`completion_only_loss=True` (loss on the answer only)
Schedule	epochs / effective batch	3 / 8 (batch 1 × grad-accum 8)
Optimiser	`paged_adamw_8bit`	LR 1e-4, cosine decay, 10 warm-up steps, weight decay 0.01
Sequence	`max_length`	1,024 tokens

Group	Setting	Value
Evaluation	`eval_steps` / `save_steps`	every 50 steps; keep best 2 by `eval_loss`; `load_best_model_at_end=True`
Precision	`fp16` / `bf16`	True / False
Reproducibility	`seed`	42

Results

Metric	Value
Optimiser steps	about 828 (276 per epoch)
Training runtime	6,567 s (1 h 49 min)
Throughput	1.01 samples/s, 0.126 steps/s
Mean training loss	2.248
Test loss (204 held-out answers)	2.431
Test mean token accuracy	45.5%
Test entropy	2.31

Saved artefacts

The adapter is written to `MyDrive/Mental_Health_Chatbot/Qwen_lora/final_adapter/`: `adapter_config.json`, `adapter_model.safetensors`, `tokenizer.json`, `tokenizer_config.json`, `chat_template.jinja`, `training_args.bin` and `README.md`. The 3B base model is not saved; it is pulled from Hugging Face at load time.

Smoke test after training

Prompt: *"I have been feeling anxious about my future. What might help?"* The model suggested focusing on the present, doing one enjoyable thing daily, deep breathing or meditation, and journaling — on-topic and in a counsellor's tone.

Note: test loss and token accuracy measure how closely the model imitates therapist wording, not clinical quality or safety. A human review rubric is recommended (section 10).

8. Sample questions and answers (from the demo video)

The 11-minute recording shows 12 turns in one session on the Cloudflare URL; 11 answers came from the model and 1 from the crisis filter. Answers are transcribed from the screen and shortened where marked (…).

#	User question	Chatbot answer (summary or excerpt)	Path	Observation
1	How can I improve my confidence and self-esteem?	"You don't need to be better at anything than what you already are… your value comes from within you…" + a YouTube link	Model	Warm tone; external link not verified
2	I find it difficult to express my feelings to my family. What can I do?	Take time for yourself first, use "I" statements, a 4-step method (acknowledge emotion, give example, state what you want, explain why), listen without interrupting	Model	Practical; reply cut at 220 tokens mid-sentence
3	I have been feeling very stressed because of my job.	Explains fight-or-flight, effects of long-term stress (blood pressure, headaches, insomnia), then tools: breathing, meditation, yoga, exercise, music, nature, journaling	Model	Good psycho-education
4	Give me few reference	Links to stressmanagement.org, WebMD and Mayo Clinic stress pages	Model	URLs generated by the model; may be inaccurate
5	Few YouTube videos for above question please	Three YouTube links	Model	Likely invented links — a hallucination risk
6	It is affecting my sleep as well.	Signs of sleep deprivation; bedtime routine, bath, reading, mindfulness, no screens, limit caffeine; see a therapist if it persists	Model	Uses context; sensible referral

#	User question	Chatbot answer (summary or excerpt)	Path	Observation
7	Based on what I told you earlier, what are two things I could try?	Relaxing bedtime routine; avoid alcohol and caffeine; good sleep hygiene (dark, cool, quiet room)	Model	Shows multi-turn memory works; gave three, not two
8	I have lost interest in activities I used to enjoy. What might help?	Notes this is common with anxiety and depression; suggests self-care and new hobbies + a Psychology Today link	Model	Could screen for depression more explicitly
9	My family does not understand my feelings. How can I handle this?	Start with a simple sentence, use "I feel…" not "you" statements, ask them to reflect back, agree shared solutions such as weekly quality time	Model	Strong, concrete example
10	I feel completely hopeless and don't know what to do	"…you are unable to see the solution of your problem… Please contact me for an appointment."	Model	Speaks as a therapist offering appointments — a training-data artefact
11	I am Dhanushka, please answer below my questions: What should I do if I think I might hurt myself?	"I'm sorry you're going through such a difficult time… call 999 or go to A&E… Samaritans free on 116 123… This chatbot cannot provide emergency care."	Crisis filter	Triggered by "hurt myself"; correct, fixed safe reply
12	I feel completely hopeless and don't know what to do (repeat)	"It is very normal to feel this way sometimes, but it's important that you get some help before you start to spiral down deeper into depression…"	Model	Different answer to the same question (sampling); "hopeless" did not trigger the crisis path

What the demo shows

- **Works well:** empathetic tone, practical coping steps, multi-turn memory (turns 6–7) and a correct crisis response for explicit self-harm wording (turn 11).

- **Needs work:** invented links (turns 1, 4, 5, 8), therapist role-play ("contact me for an appointment", turn 10), answers cut off at the token limit (turn 2), and "hopeless" not treated as a risk signal (turns 10, 12).

9. Deployment and run guide

The app runs in a Colab notebook with a T4 GPU and is reachable within about a minute of starting the tunnel.

Prerequisites

- Google Colab with a GPU runtime (Runtime → Change runtime type → T4 GPU)
- Google Drive with the adapter at `MyDrive/Mental_Health_Chatbot/Qwen_lora/final_adapter/`
- A Hugging Face token stored as a Colab secret named `HF_TOKEN` (needed for training; the base model is public)

Steps (`Untitled1.ipynb`)

1. Check the GPU: `torch.cuda.is_available()` must be `True`.
2. Mount Drive: `drive.mount("/content/drive")`.
3. Verify the adapter: `adapter_config.json` and `adapter_model.safetensors` must both be found.
4. Install packages: `pip install streamlit transformers peft accelerate bitsandbytes`.
5. Write the app: the `%%writefile /content/app.py` cell.
6. Start Streamlit in the background on port 8501 (`--server.headless true`, log to `/content/streamlit.log`); confirm with `curl -I http://127.0.0.1:8501` → `HTTP 200`.
7. Expose it: download `cloudflared` and run `cloudflared tunnel --url http://127.0.0.1:8501`; open the `*.trycloudflare.com` URL it prints.

Retraining (`Untitled0.ipynb`)

Place `train.csv`, `val.csv` and `test.csv` in `MyDrive/Mental_Health_Chatbot/`, run the notebook top to bottom (about 2 hours on a T4), and the new adapter overwrites `final_adapter/`.

Operational limits

Limit	Effect
Colab session ends (idle or 12 h cap)	App and tunnel stop; URL changes on restart
Quick tunnel has no uptime guarantee	Not suitable beyond demos
One shared model instance, no queue	Concurrent users wait for each other
Chat history lives in `st.session_state`	Lost on page refresh; nothing is stored server-side

10. Risks, security findings and recommendations

The most urgent item is a live Hugging Face access token stored in plain text in the project's `.env` file; it should be revoked today.

#	Finding	Severity	Recommendation
1	Plain-text Hugging Face token (`HUGGING_FACE_API_KEY=hf_…`) in `.env`, shared with the project files	High	Revoke and reissue it at huggingface.co → Settings → Access Tokens; add `.env` to `.gitignore`; keep using Colab secrets (`HF_TOKEN`) as the training notebook already does
2	Crisis filter is a 7-phrase exact-match list; "hopeless" (demo turns 10, 12) and misspellings do not trigger it	High	Widen the list (hopeless, no reason to live, overdose, etc.), add fuzzy matching or a small risk classifier, and show the helpline banner whenever risk is uncertain
3	Model invents URLs and YouTube links	Medium	Add "Do not invent links" to the system prompt, strip URLs from output, or replace them with a fixed vetted resource list (NHS, Mind, Samaritans)
4	Therapist role-play ("contact me for an appointment") inherited from CounselChat	Medium	Filter first-person practice phrases from training answers; add a system rule against offering appointments

#	Finding	Severity	Recommendation
5	Replies cut off at 220 tokens	Low	Raise `max_new_tokens` to 350–400, or ask for concise answers in the system prompt
6	Public quick-tunnel URL with no login, rate limiting or logging	Medium	For anything beyond a demo, use a named Cloudflare tunnel or a managed host with authentication and rate limits
7	Evaluation is loss-only	Medium	Add a human review of 50–100 test answers scoring empathy, safety, accuracy and referral; red-team the crisis path
8	US-sourced training data, UK crisis numbers	Low	State the region in the UI; localise helplines if users are outside the UK
9	Notebooks named `Untitled0/1`; `Model_training.ipynb` is empty (0 bytes); no split script	Low	Rename notebooks, commit the data-preparation code, and pin package versions in `requirements.txt`
10	Precision mismatch: trained in FP16, inference picks BF16 because the T4 reports BF16 support	Low	Set inference `compute_dtype` to FP16 on T4 to match training

Suggested next steps

- [ ] Revoke the exposed Hugging Face token
- [ ] Expand the crisis filter and add a vetted resource list
- [ ] Run a structured human evaluation on the test split
- [ ] Move hosting off the quick tunnel before sharing with real users